

Life Long Learning

End-to-End case study

Sequential Models in NLP

Sarcasm Detection

Description

Past studies in Sarcasm Detection mostly make use of Twitter datasets collected using hashtag based supervision but such datasets are noisy in terms of labels and language. Furthermore, many tweets are replies to other tweets and detecting sarcasm in these requires the availability of contextual tweets.

In this hands-on project, the goal is to build a model to detect whether a sentence is sarcastic or not, using Bidirectional LSTMs.

Dataset

News Headlines dataset for Sarcasm Detection

The dataset is collected from two news websites, theonion.com and huffingtonpost.com. This new dataset has the following advantages over the existing Twitter datasets:

Since news headlines are written by professionals in a formal manner, there are no spelling mistakes and informal usage. This reduces the sparsity and also increases the chance of finding pre-trained embeddings. Furthermore, since the sole purpose of TheOnion is to publish sarcastic news, we get high-quality labels with much less noise as compared to Twitter datasets.

Unlike tweets that reply to other tweets, the news headlines obtained are self-contained. This would help us in teasing apart the real sarcastic elements.

Content

Each record consists of three attributes:

- is_sarcastic: 1 if the record is sarcastic otherwise 0
- headline: the headline of the news article
- article_link: link to the original news article. Useful in collecting supplementary data

Reference

Sarcasm_Headlines_Dataset.json

Implementation steps

Instructions to perform all the below steps are mentioned in the question notebook with the respective marks.

1. Read and explore the data
2. Drop one column
3. Get length of each sentence
4. Define parameters
5. Get indices for words
6. Create features and labels
7. Get vocab size
8. Create a weight matrix using GloVe embeddings
9. Define and compile a Bidirectional LSTM model
10. Fit the model and check the validation accuracy

Answer:

Objectives:

The **primary outcome** of this sarcasm detection project is a **trained deep learning model** that can classify whether a given news headline is sarcastic (1) or not (0). Here's a breakdown of the expected outputs:

1. Trained Bidirectional LSTM Model

- A **sequential NLP model** built using:
 - **Embedding Layer**: Initialized with **pre-trained GloVe word vectors** (e.g., glove.6B.50d.txt or 200d).
 - **Bidirectional LSTM**: Captures contextual sarcasm cues from both forward and backward sequences.
 - **Dense Layers**: For classification with a **sigmoid output** (binary: 0 or 1).
- **Saved Model**: Can be exported for deployment (e.g., .h5 or TensorFlow SavedModel format).

2. Validation Accuracy

- The model's performance is evaluated on a **held-out test set** (typically 20% of the data).
- Expected **validation accuracy**: ~75–85% (varies based on hyperparameters and data quality).
- Metrics reported:
 - **Accuracy**: Overall correctness.
 - **Precision/Recall**: Especially important for imbalanced datasets (if sarcastic/non-sarcastic ratios are skewed).
 - **Confusion Matrix**: Shows false positives/negatives.

3. Key Insights from the Project

1. Data Quality Advantage:

- The **Onion (sarcastic) vs. Huffington Post (non-sarcastic)** dataset is cleaner than noisy Twitter data.
- Headlines are self-contained, avoiding contextual dependencies.

2. Model Strengths:

- **Bidirectional LSTMs** outperform simpler models (e.g., vanilla RNNs) by capturing sarcasm cues like:
 - **Irony:** E.g., "What a great day" during a storm.
 - **Unexpected contrasts:** E.g., "Finally, a healthy fast food burger."
- **GloVe embeddings** help generalize rare words (e.g., "breathtakingly awful").

3. Limitations:

- May struggle with subtle sarcasm or cultural references.
- Requires retraining for domain adaptation (e.g., social media slang).

4. Practical Applications

- **Automated Moderation:** Flag sarcastic headlines in news aggregators.
- **Social Media Analysis:** Extend to tweets/comments (with additional training).
- **Chatbots/Assistants:** Improve tone detection in user inputs.

Deliverables

Code: In Google colab

Data preprocessing (tokenization, padding).

Model training/evaluation.

Model Files: Trained weights + architecture.

Performance Report: Accuracy/loss curves, test metrics.

```
[ ] #Load data
```

Double-click (or enter) to edit

```
[ ] import json
import pandas as pd
```

```
[ ] #load the JSON data
```

```
▶ with open('/content/Sarcasm_Headlines_Dataset.json','r') as f:
    data =[json.loads(line) for line in f]

#convert to dataframe

df= pd.DataFrame(data)
```

```
[ ] df.head()
```

```
is_sarcastic      headline      article_link
0      1  thirtysomething scientists unveil doomsday clo...  https://www.theonion.com/thirtysomething-scienc...
1      0    dem rep. totally nails why congress is falling...  https://www.huffingtonpost.com/entry/donna-edw...
2      0    eat your veggies: 9 deliciously different recipes  https://www.huffingtonpost.com/entry/eat-your-...
3      1  inclement weather prevents liar from getting t...  https://local.theonion.com/inclement-weather-p...
4      1  mother comes pretty close to using word 'strea...  https://www.theonion.com/mother-comes-pretty-c...

] #2.Drop artical_link from dataset

df= df.drop(['article_link'],axis=1)

] df.head()
```

	is_sarcastic	headline
0	1	thirtysomething scientists unveil doomsday clo...
1	0	dem rep. totally nails why congress is falling...
2	0	eat your veggies: 9 deliciously different recipes

```
[ ] df.columns

Index(['is_sarcastic', 'headline'], dtype='object')

] #3.Get the length of each headline and add a column for that

[ ] df['headline_length']=df['headline'].apply(lambda x:len(x.split()))

df['headline_length']
```

	headline_length
0	8
1	13
2	7
3	8
4	9

```
[ ] #4.Initialize parameter values
```

```
[ ] max_features =10000    #no.of words consider from tokenizer  
    maxlen =25            #max length of each sentence  
    embedding_size =200    #size of embedding vector
```

```
[ ] #5.Apply tokenizer and get indices for words
```

```
[ ] from tensorflow.keras.preprocessing.text import Tokenizer  
    from tensorflow.keras.preprocessing.sequence import pad_sequences
```



```
Tokenizer =Tokenizer(num_words=max_features, oov_token='OOV')  
Tokenizer.fit_on_texts(df['headline'])  
sequences = Tokenizer.texts_to_sequences(df['headline'])
```

```
[ ] sequences
```



```
[[1, 355, 3167, 7474, 2644, 3, 661, 1119],  
 [7475, 1775, 758, 3168, 47, 239, 11, 1844, 1048, 8, 1528, 2154, 1845],  
 [863, 33, 1, 262, 1, 647, 1484],  
 [1, 1594, 6418, 5665, 16, 143, 2, 152],
```

```
[ ] df.head(1)
```

```

is_sarcastic      headline      headline_length
0            1  thirtysomething scientists unveil doomsday clo...      8

```

```
[ ] #6.Pad sequences and prepare target array
```

```

[ ] x= pad_sequences(sequences,maxlen=maxlen,padding='post',truncating='post')
    y=df['is_sarcastic'].values

```

```
[ ] x
```

```

array([[ 1, 355, 3167, ..., 0, 0, 0],
       [7475, 1775, 758, ..., 0, 0, 0],
       [ 863, 33, 1, ..., 0, 0, 0],
       ...,
       [ 4, 100, 629, ..., 0, 0, 0],
       [1870, 1313, 3317, ..., 0, 0, 0],
       [ 217, 3283, 21, ..., 0, 0, 0]], dtype=int32)

```

```
[ ] y
```

```
[ ] #7.Vocab mapping and setting number of words
```

```
[ ] word_index = Tokenizer.word_index  
    Vocab_size = len(word_index)+1
```



word_index



```
'today': 541,  
'dream': 542,  
'fan': 543,  
'feel': 544,  
'ted': 545,  
'crisis': 546,  
'sleep': 547,  
'letter': 548,  
'mark': 549,  
'word': 550,  
'cat': 551,  
'i'm': 552,  
'wins': 553,  
'administration': 554,  
'south': 555,  
'director': 556,  
'she's': 557,  
'hidden': 558
```

```
[ ] Vocab_size
```

```
⇒ 30886
```

```
[ ] #Load GloVe word Embeddings
```

```
import numpy as np
embeddings_index = {}
with open('/content/glove.6B.50d.txt', encoding='utf-8') as f:
    for line in f:
        values = line.split()
        word = values[0]
        coefs = np.asarray(values[1:], dtype='float32')
        embeddings_index[word] = coefs
```

```
[ ] coefs
```

```
⇒ array([-0.43264, -1.0114 ,  0.69846, -0.48718, -0.63683,  0.18958],
        dtype=float32)
```

```
[ ] embeddings_index[word]
```

```
[ ] embeddings_index[word]
```

```
⇒ array([-0.43264, -1.0114 ,  0.69846, -0.48718, -0.63683,  0.18958],
        dtype=float32)
```

```
[ ] #9.Create embedding matrix
```

```
[ ] #9.Create embedding matrix
embedding_size = 50 # Change embedding_size to 50 to match GloVe embeddings
embedding_matrix = np.zeros((Vocab_size, embedding_size))
for word, i in word_index.items():
    if i < max_features:
        embedding_vector = embeddings_index.get(word)
        if embedding_vector is not None:
            embedding_matrix[i] = embedding_vector
```

```
[ ] #10.Define Model
```

```
[ ] #9.Create embedding matrix
embedding_size = 50 # Change embedding_size to 50 to match GloVe embeddings
embedding_matrix = np.zeros((max_features, embedding_size)) # Change Vocab size to max features
```



```
[ ] #9.Create embedding matrix
embedding_size = 50 # Change embedding_size to 50 to match GloVe embeddings
embedding_matrix = np.zeros((max_features, embedding_size)) # Change Vocab_size to max_features
for word, i in word_index.items():
    if i < max_features:
        embedding_vector = embeddings_index.get(word)
        if embedding_vector is not None:
            embedding_matrix[i] = embedding_vector
```

```
[ ] #11. Compile the model
```

```
▶ #10.Define Model
from tensorflow import keras
from tensorflow.keras.layers import Embedding, LSTM, Dense

model = keras.Sequential([
    Embedding(max_features, embedding_size, weights=[embedding_matrix], input_length=maxlen, trainable=False),
    LSTM(128),
    Dense(1, activation='sigmoid') # For binary classification (sarcastic/not sarcastic)
])
```

ommands + Code + Text

Connect ▼

```
[ ] Dense(1, activation='sigmoid') # For binary classification (sarcastic/not sarcastic)
])

#11. Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

 /usr/local/lib/python3.11/dist-packages/keras/src/layers/core/embedding.py:90: UserWarning: Argument `input\_length` is deprecated. Just remove it. warnings.warn()

```
[ ] #12. Fit the model
```

```
▶ from sklearn.model_selection import train_test_split

# Change 'X' to 'x' to use the padded sequences
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)

history = model.fit(X_train, y_train,
                    epochs=10,
                    batch_size=32,
                    validation_data=(X_test, y_test))
```

```
Epoch 1/10
716/716 ————— 28s 35ms/step - accuracy: 0.6717 - loss: 0.5951 - val_accuracy: 0.8068 - val_loss: 0.4212
Epoch 2/10
716/716 ————— 41s 36ms/step - accuracy: 0.8050 - loss: 0.4255 - val_accuracy: 0.8145 - val_loss: 0.4055
Epoch 3/10
716/716 ————— 39s 33ms/step - accuracy: 0.8373 - loss: 0.3760 - val_accuracy: 0.8347 - val_loss: 0.3631
Epoch 4/10
716/716 ————— 43s 35ms/step - accuracy: 0.8506 - loss: 0.3397 - val_accuracy: 0.8372 - val_loss: 0.3634
Epoch 5/10
716/716 ————— 40s 34ms/step - accuracy: 0.8700 - loss: 0.3022 - val_accuracy: 0.8531 - val_loss: 0.3370
Epoch 6/10
716/716 ————— 26s 36ms/step - accuracy: 0.8795 - loss: 0.2802 - val_accuracy: 0.8567 - val_loss: 0.3336
Epoch 7/10
716/716 ————— 26s 36ms/step - accuracy: 0.8939 - loss: 0.2547 - val_accuracy: 0.8503 - val_loss: 0.3596
Epoch 8/10
716/716 ————— 25s 35ms/step - accuracy: 0.9088 - loss: 0.2206 - val_accuracy: 0.8529 - val_loss: 0.3490
Epoch 9/10
716/716 ————— 25s 35ms/step - accuracy: 0.9223 - loss: 0.1993 - val_accuracy: 0.8510 - val_loss: 0.3775
Epoch 10/10
716/716 ————— 41s 36ms/step - accuracy: 0.9302 - loss: 0.1779 - val_accuracy: 0.8501 - val_loss: 0.3735
```

```
[ ]
history
```

```
<keras.src.callbacks.history.History at 0x7cbc697e8cd0>
```